

# Final Report

Ziyu(Caroline) Xu

April 2, 2025

## Introduction

This project focuses on building an effective fish detection model using the Caltech Fish Counting Dataset (CFC-DAOD). One of the key challenges in this task is domain adaptation — specifically, training the model on one location (Kenai Left Bank) and ensuring it performs well on a different location (Kenai Channel), where lighting, background, and fish appearance may vary.

To tackle this, I began with exploratory data analysis to understand the size of the fish, the complexity of the backgrounds, and the distribution of bounding boxes. These insights helped shape the modeling decisions. Given the need for speed, flexibility, and strong small-object detection capabilities, I chose YOLO (You Only Look Once) as the main detection model. This report outlines the analysis and reasoning behind that choice, as well as the steps taken to optimize the model for better generalization across domains.

## Data Analysis

### Dataset Overview

The dataset follows the COCO annotation format which includes images, annotations, and categories. The images have metadata such as file name, image ID, image height and image width. The annotations have annotation about bounding box coordinates (x, y, width, height) for detected fish. The categories for both train and test contain only one fish species which the category id is 1 and the name is 'fish'. Table 1 shows the training set consists of 76619 images with 132010 annotations, while the test set includes 13090 images with 41761 annotations.

| Dataset        | Images | Annotations | Categories |
|----------------|--------|-------------|------------|
| Source (Train) | 76619  | 132010      | 1          |
| Target (Test)  | 13090  | 41761       | 1          |

Table 1: Summary of the dataset composition, including the number of images, annotations, and categories for the training (Source) and test (Target) sets.

### Target Size Distribution

Table 2 provides a summary of the bounding box dimensions for fish instances in the dataset. The width of the bounding boxes ranges from 2 to 214 pixels, with an average of 68.55 pixels. The height ranges from 1 to 164 pixels, with an average of 22.21 pixels. Bounding box areas range from 6 pixels<sup>2</sup> to 20,424 pixels<sup>2</sup>, with an average of 1,671.21 pixels<sup>2</sup>. This variation in size highlights the presence of both very small and relatively larger fish in the dataset, requiring a model capable of detecting small objects effectively.

Figure 1 provides further insight into the size distribution of bounding boxes by illustrating their area ratios relative to the entire image. The histogram shows that the majority of bounding boxes occupy a small fraction of the image, with most area ratios clustering below 0.01. The red dashed line at 5% area ratio highlights that almost all fish are small object.

To complement this analysis, Figure 2 presents a scatter plot of bounding box width versus height, with color representing the bounding box area. This visualization reinforces the observation that most fish have narrow, elongated bounding boxes (low height-to-width ratios), and a significant number of fish have very

| Metric                      | Minimum | Maximum | Mean    |
|-----------------------------|---------|---------|---------|
| Width (pixels)              | 2.0     | 214.0   | 68.55   |
| Height (pixels)             | 1.0     | 164.0   | 22.21   |
| Area (pixels <sup>2</sup> ) | 6.0     | 20424.0 | 1671.21 |

Table 2: Summary of Bounding Box Dimensions in the Dataset

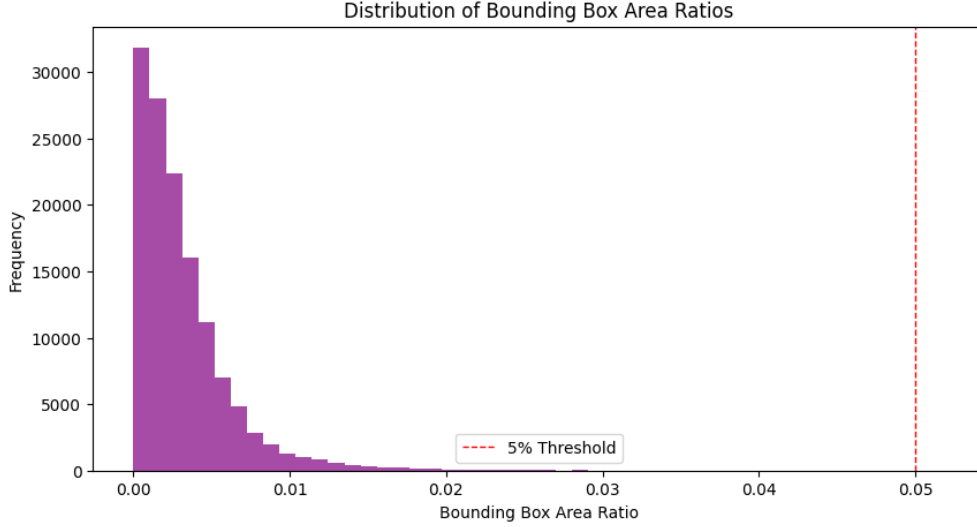


Figure 1: Distribution of Bounding Box Area Ratios in the Dataset

small bounding boxes in both dimensions. The color gradient further highlights the concentration of small objects in the dataset.

Given these findings, it’s clear that detecting small fish requires preserving fine image details. Since most bounding boxes are very small, increasing the input resolution (for example, from  $512 \times 512$  to  $1024 \times 1024$ ) helps the model capture more information and improves its ability to detect small targets. This adjustment proved to be an effective strategy for enhancing performance, especially in complex scenes with noisy or cluttered backgrounds.

### Background Complexity Analysis

The background complexity of the dataset was analyzed using color complexity scores. It measures the variation in hue and saturation across images. The average color complexity score is 34,077.68, with a standard deviation of 23,909.92. This indicates a large difference in background characteristics. This high variability suggests that the dataset contains a mix of images with simple and highly complex backgrounds. Figure 3 presents the distribution of color complexity scores. The histogram shows a multi-modal distribution, with distinct peaks at different complexity levels. The first peak, located at a lower complexity range, likely corresponds to images with relatively uniform backgrounds, such as clear water conditions. The second and third peaks, appearing at higher complexity scores, indicate the presence of images with more complicated backgrounds. It might includes underwater vegetation, varying lighting conditions, or reflections.

The dataset’s diverse background complexity presents a significant challenge for object detection models, as training on simpler scenes may limit generalization to visually cluttered environments. To address this, future experiments will incorporate data augmentation techniques such as mosaic, mixup, and random erasing. These methods are expected to enhance the model’s ability to distinguish fish from background distractions like bubbles, reflections, and underwater plants.

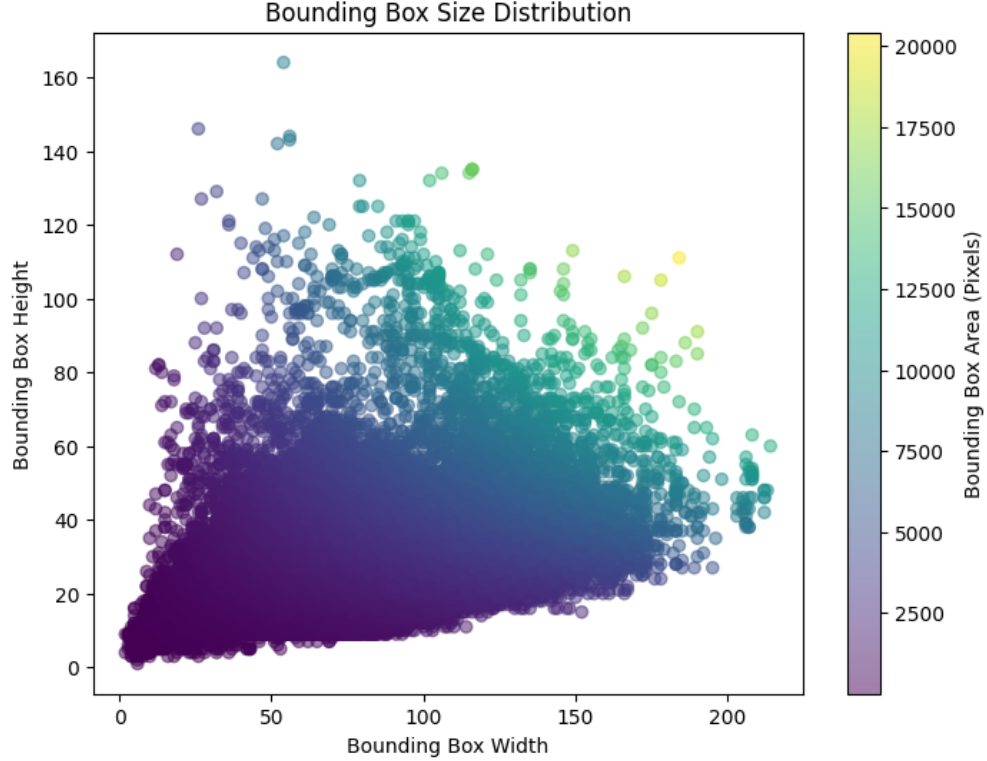


Figure 2: Bounding Box Size Distribution in the Dataset

### Texture Complexity Analysis

The texture complexity of the dataset was assessed using edge density, which quantifies the proportion of edge pixels in each image. The analysis demonstrates an average edge density of 0.6756, with a standard deviation of 0.5590. It indicates significant difference in texture across images. This variation suggests that there are some images feature relatively smooth backgrounds, but others contain a high degree of texture. It might due to factors such as underwater vegetation, water surface disturbances, or image noise. Figure 4 illustrates the distribution of edge density scores. The histogram shows a right-skewed distribution, where most images have low edge density. A smaller subset exhibits significantly higher values.

High texture complexity can lead to false positives because strong background edges may be misinterpreted as fish. However, low texture complexity may make it harder to distinguish fish from their surroundings. To improve robustness, Gaussian blur can help reduce background noise in high-texture images. CutMix augmentation can enhance generalization.

### Target Density Analysis

The distribution of fish across images was analyzed to assess the likelihood of object overlap and potential occlusion challenges for the detection model. The dataset contains a total of 132,010 fish annotations across 76,619 images. It results in an average fish density of 1.72 fish per image. This relatively low density suggests that most images contain only one or two fish. It reduces the probability of overlap. Additionally, the average distance between fish instances is 246.84 pixels. It could further indicate that fish are generally well-separated within images. Given these findings, the dataset does not present significant challenges related to occlusion. Since fish are not frequently overlapping, non-maximum suppression (NMS) in YOLO can remain at its default setting.

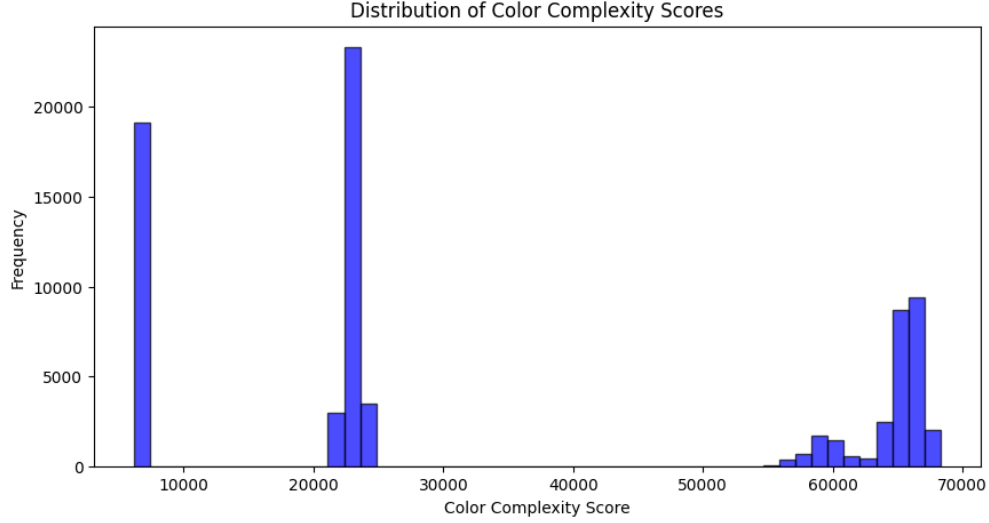


Figure 3: Distribution of Color Complexity Scores in the Dataset

## Method

The methodology for this study focuses on developing an optimized fish detection model using YOLO. The main challenges include small object detection, complex underwater backgrounds, and visual domain shifts. This section outlines the rationale behind model selection, as well as experimental settings such as input resolution, batch size, and augmentation techniques aimed at improving model performance and generalizability.

### Model Selection: Why YOLO?

YOLO was selected as the primary detection model for its efficiency in real-time inference and its strong performance on small-object detection tasks. Unlike two-stage detectors such as Faster R-CNN, YOLO processes the image in a single pass, making it computationally lightweight and suitable for experiments on limited resources. Its built-in support for augmentation strategies and easy configurability made it ideal for iterative experimentation on challenging fish datasets.

### Optimizations for Small Object Detection

To improve YOLO’s performance on small fish in the dataset, the following configurations were explored:

- **Input Resolution:** Different input sizes were tested, including 512 and 1024. Larger image resolutions helped the model detect smaller fish more accurately by preserving finer spatial details, though training time increased significantly.
- **Batch Size:** Training was conducted with batch sizes of 4, 8, and 16. A larger batch size stabilized training and improved accuracy, but required more memory.
- **Data Augmentation:** Several augmentation methods were tested individually:
  - **Mosaic:** Combines four training images into one, helping the model generalize to different contexts.
  - **Mixup:** Blends two images and their labels to introduce more visual variability.
  - **Copy-Paste:** Inserts fish from one image into another, simulating occlusion and fish overlap.
  - **Erasing:** Randomly masks part of the image to force the model to rely on multiple features.

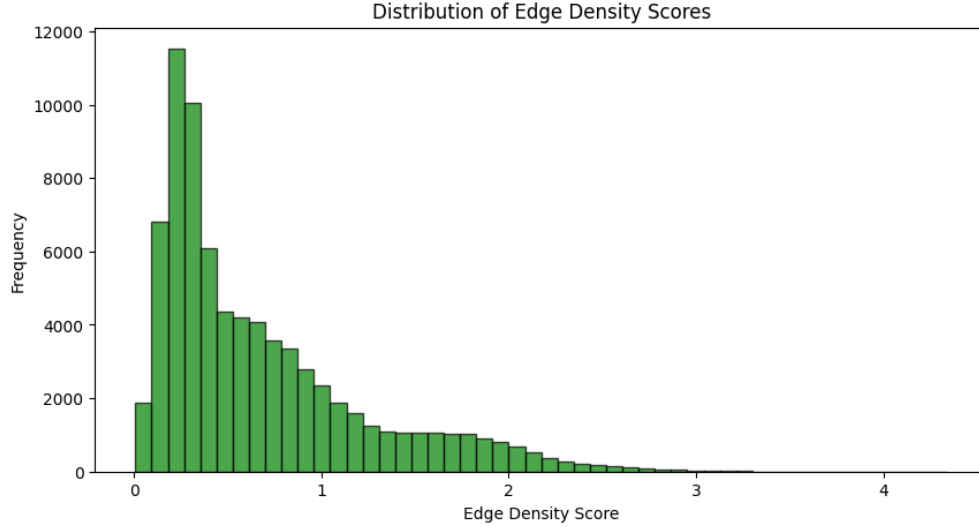


Figure 4: Distribution of Edge Density Scores in the Dataset

## Results

In order to validate the effect of different settings on small object detection, a number of training experiments were conducted with differences in image resolution, batch size, and augmentation methods. The performance on key metrics: mean Average Precision at IoU 0.5 (mAP@50), precision, recall, and loss is compared in the table below:

| Configuration         | mAP@50       | Precision    | Recall       | Val Loss    |
|-----------------------|--------------|--------------|--------------|-------------|
| Batch size = 16       | <b>0.592</b> | <b>0.732</b> | 0.508        | <b>4.96</b> |
| Mixup = 0.3           | 0.585        | 0.689        | <b>0.534</b> | 5.08        |
| Batch size = 8        | 0.574        | 0.684        | 0.528        | 5.14        |
| 1024 + Batch = 8      | 0.573        | 0.663        | 0.521        | 5.18        |
| Mixup = 0.5           | 0.573        | 0.711        | 0.496        | 5.09        |
| Mosaic = 0.7          | 0.573        | 0.726        | 0.457        | 5.15        |
| Erasing = 0.0         | 0.563        | 0.682        | 0.509        | 5.10        |
| Image Size = 1024     | 0.557        | 0.615        | 0.505        | 5.29        |
| Erasing = 0.6         | 0.555        | 0.691        | 0.481        | 5.20        |
| Base (512, Batch = 4) | 0.542        | 0.660        | 0.460        | 5.04        |
| Copy-paste = 0.5      | 0.539        | 0.629        | 0.519        | 5.19        |
| Mosaic = 0.5          | 0.526        | 0.675        | 0.464        | 5.22        |

Table 3: Validation performance of different YOLO configurations on fish detection.

The results clearly show the effect of various training settings on accuracy and generalization. Among all settings, batch size 16 achieved the highest mAP@50 score of 0.592 and precision of 0.732 and the lowest validation loss (4.96), showing that it was the most stable and effective setting. Larger batch sizes likely averaged out gradient updates and prevented overfitting. Mixup augmentation with a factor of 0.3 achieved the highest recall (0.534), i.e., it was especially good at reducing missed detections. This configuration had competitive precision and loss, making it a strong candidate for real-world deployment where it is critical to catch as many fish as possible. Though image size 1024 outperformed the baseline model, it also led to higher validation loss (5.29), perhaps due to the higher complexity and computational cost. Mosaic augmentation at 0.7 and 0.5 had mixed impacts: precision improved, but recall decreased, and validation loss was still above 5.1. Erasing augmentation yielded modest improvements. Disabling erasing (erasing=0.0) yielded higher recall than enabling it (erasing=0.6), though the latter achieved slightly better precision.

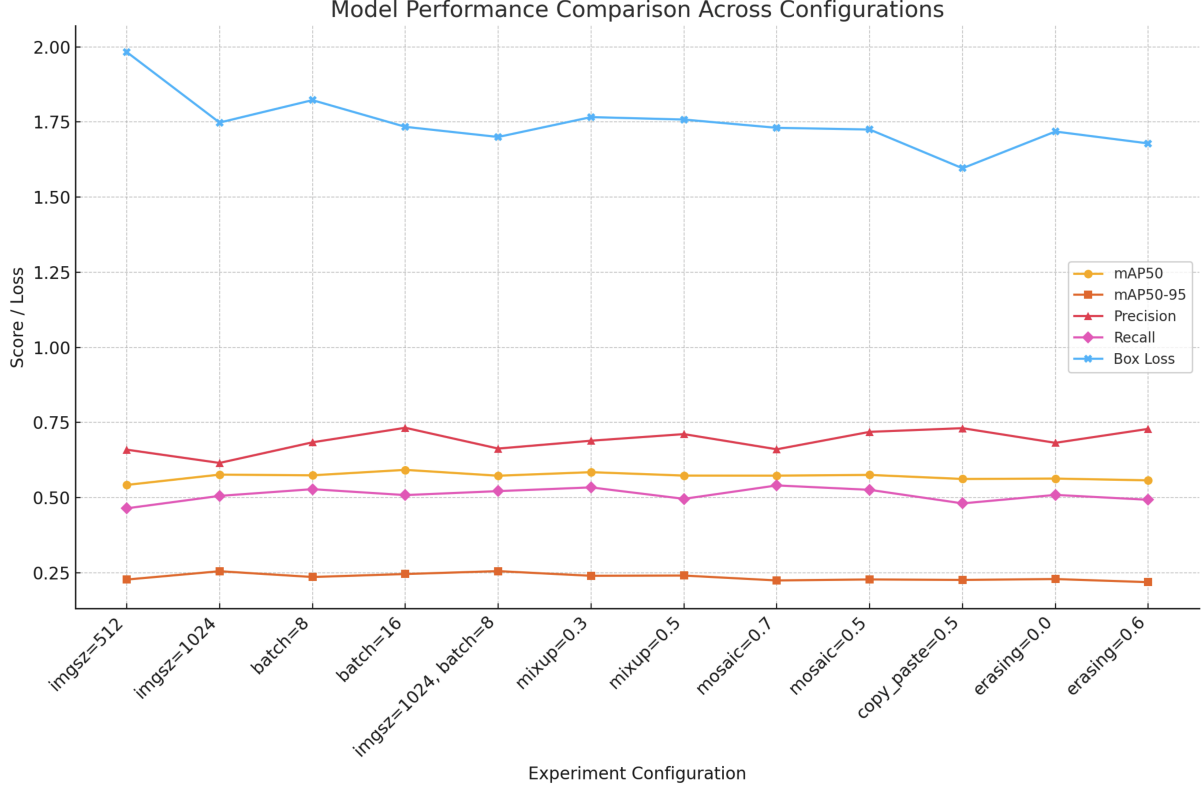


Figure 5: Comparison of key performance metrics across different YOLO training configurations.

The baseline configuration — no advanced augmentations and no resolution changes — understandably performed worse on all metrics. This shows the conclusion that augmentations and precise tuning are crucial to this small-object detection task.

## Conclusion

This project is an investigation into small object detection with YOLO on the Caltech Fish Counting Dataset. The task was discovered to be especially difficult due to several reasons: the extremely small size of the fish relative to the image resolution, the complex and noisy underwater backgrounds, and significant domain shifts between different subsets.

To address these challenges, a series of controlled experiments were applied. Those helped to examine different training configurations. Modifications included modifying input image resolution (e.g.,  $512 \times 512$  vs.  $1024 \times 1024$ ), batch sizes (4, 8, 16), and applying varied data augmentation techniques such as mixup, mosaic, copy-paste, and erasing. The most notable observations are the consistent improvement in recall and precision with high batch size and mosaic augmentation. Mixup and copy-paste further improved performance on complex scenes through increased training data diversity. High-resolution images also appeared to stabilize training, but it had a greater computational cost. Of all the test configurations, the best arrangement achieved a mAP@50 score of 0.59 with associated increases in precision (to 0.73) and recall (0.53), demonstrating the worth of these modified approaches.

While YOLO demonstrated remarkable flexibility and velocity, the model still struggled with overlapping fish, low-contrast environments, and background-generated false positives under such features as vegetation or shadows. These errors also suggest potential methods for enhancement: increased training intervals, exploring even larger YOLO variants (e.g., YOLOv8-L), and creating transformer-based detectors that can potentially better capture contextual patterns in underwater settings.

In short, this project reinforces the importance of experimental iteration, domain-aware augmentation, and resolution tuning in fine-grained object detection tasks. The findings contribute to the understanding of how modern detection frameworks can be repurposed to serve scientific applications with small, cluttered, and visually unclear targets—like fish in the wild.

## References

- [1] Ultralytics. (2024). *Ultralytics YOLO Documentation*. <https://docs.ultralytics.com>
- [2] Kay, J., Haucke, T., Stathatos, S., Deng, S., Young, E., Perona, P., Beery, S., & Van Horn, G. (2025). *Align and Distill: Unifying and Improving Domain Adaptive Object Detection*. arXiv. Retrieved from <https://doi.org/10.48550/arXiv.2403.12029>
- [3] Kay, J., Kulits, P., Stathatos, S., Deng, S., Young, E., Beery, S., Van Horn, G., & Perona, P. (2022). *The Caltech Fish Counting Dataset: A Benchmark for Multiple-Object Tracking and Counting*. arXiv. Retrieved from <https://doi.org/10.48550/arXiv.2207.09295>